

**KARADENİZ TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ BİLGİSAYAR
MÜHENDİSLİĞİ BÖLÜMÜ**



PROJENİN KONUSU:

2D metroidvania türü bilgisayar tabanlı macera odaklı , çocuk ve yetişkin herkesin oynayabileceği oyun geliştirilecektir.

Yazılım Mühendisliği Öneri Raporu

425492 Alaattin Uysal

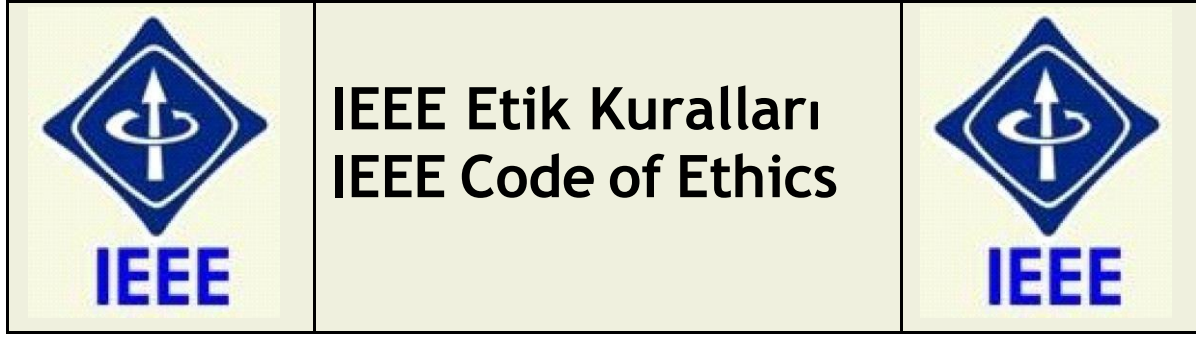
425429 Eren Kuşdil

425472 Özer Koca

425468 Alperen Altunel

425500 Berat Kayar

2024-2025 BAHAR DÖNEMİ



Mesleğime karşı şahsi sorumluluğumu kabul ederek, hizmet ettiğim toplumlara ve üyelerine en yüksek etik ve mesleki davranışta bulunmaya söz verdiğimi ve aşağıdaki etik kurallarını kabul ettiğimi ifade ederim:

- Kamu güvenliği, sağlığı ve refahı ile uyumlu kararlar vermenin sorumluluğunu kabul etmek ve kamu veya çevreyi tehdit edebilecek faktörleri derhal açıklamak;
- Mümkün olabilecek çıkar çatışması, ister gerçekten var olması isterse sadece algı olması, durumlarından kaçınmak. Çıkar çatışması olması durumunda, etkilenen taraflara durumu bildirmek;
- Mevcut verilere dayalı tahminlerde ve fikir beyan etmelerde gerçekçi ve dürüst olmak;
- Her türlü rüşveti reddetmek;
- Mütenasip uygulamalarını ve muhtemel sonuçlarını gözeterek teknoloji anlayışını geliştirmek;
- Teknik yeterliliklerimizi sürdürmek ve geliştirmek, yeterli eğitim veya tecrübe olması veya işin zorluk sınırları ifade edilmesi durumunda ancak başkaları için teknolojik sorumlulukları üstlenmek;
- Teknik bir çalışma hakkında yansız bir eleştiri için uğraşmak, eleştiriye kabul etmek ve eleştiriye yapmak; hatları kabul etmek ve düzeltmek; diğer katkı sunanların emeklerini ifade etmek;
- Bütün kişilere adilane davranmak; ırk, din, cinsiyet, yaş, milliyet, cinsi tercih, cinsiyet kimliği, veya cinsiyet ifadesi üzerinden ayrımcılık yapma durumuna girişmemek;
- Yanlış veya kötü amaçlı eylemler sonucu kimsenin yaralanması, mülklerinin zarar görmesi, itibarlarının veya istihdamlarının zedelenmesi durumlarının oluşmasından kaçınmak;
- Meslektaşlara ve yardımcı personele mesleki gelişimlerinde yardımcı olmak ve onları desteklemek.

IEEE Yönetim Kurulu tarafından Ağustos 1990'da
onaylanmıştır

İÇİNDEKİLER

	Sayfa No
IEEE ETİK KURALLARI.....	I
ÖNSÖZ.....	II
İÇİNDEKİLER.....	III
1. GENEL BİLGİLER.....	4
1.1. Giriş.....	4
1.2. Problem tanımı ve projenin başlatılma gerekçesi.....	4
2.YAZILIM MÜHENDİSLİĞİ AÇISINDAN DEĞERLENDİRME.....	5
2.1. Teknik Tasarım ve Mimari Uygunluk.....	5
2.2. Geliştirme Süreci ve Metodoloji	6
2.3. Yazılım Mühendisliği Prensiplerine Uygunluk	6
2.4. Ekip Yönetimi ve Proje Uygulanabilirliği	7
2.5 Kullanılan Teknolojiler ve Uygunluk.....	7
3. ÜST MİMARİ TASARIM.....	8
3.1 Model (Oyun mantığı).....	8
3.2 View (Görselleştirme).....	8
3.3 Controller (Kontrol).....	8
4. Ekip yapısı ve görev dağılımı	9
5. Kaynakça.....	9

1-PROBLEM TANIMI VE PROJE AMACI

Problem Tanımı

Projemiz mühendislik açısından herhangi bir probleme çözüm üretmese de , çocuklar ve yetişkin bireyler için eğlenceli dakikalar ifade ediyor.

Oyun sektörünün yazılım alanında en büyük ilk 5 sektörden birisi olmasına ve ülke rezervine katkı sağlamasına rağmen ülkemizde istenilen gelişimi gösterememiş ve dünyaya kıyasla çok geride kalmıştır.

Amacımız ileride bu sektörde canlılık oluşturmak isterken, ilk takım çalışması, git kullanımı ve iş birliği ileride çalışmak istediğimiz bu sektörde başta kendimize katkı sağlamak ve belki gamejame, hackathon tarzı projelerde kod yarışmasına katılım sağlayabilmek olabilir.

5 kişilik deneyimsiz bir takım ve 3 aylık bir süre ile unity vs. gibi büyük oyun motorlarını kullanmak yerine hem program becerilemizi geliştirmek ve pygame ile de bu işin kısa sürede ve iyi dinamiklerle yapılacağını kendi çapımızda ispatlamak istedik.

Proje Amacı

2D Platform-Macera türünde , Metroidvania oyun türüdür. Tüm proje Python ve ilgili kütüphaneler (pygame, JSON, Time, random vs.) kullanılarak oyunun tasarımı 5 kişilik bir ekip ile oluşturulacaktır.

Oyunda kullanılacak mekanikler: Oyuncuların sağa-sola hareket ettiği, zıpladığı, platformları aştığı ve düşmanlarla çarpıştığı bir mekanik var. Bu, klasik "platformer" oyunlarının temel taşıdır. Enerji ve mutluluk seviyeleri, karakterin zıplama mesafesini ve saldırı gücünü etkiler; düşükse oyun zorlaşır.

Macera Unsurları: Lanetli karakterlerin bir hikaye doğrultusunda hazineye ulaşmaya çalıştığı, seçimlerle hikayenin şekillendiği bir yolculuk içeriyor. Bu da "macera" (adventure) türünü ekler. Her seviye, bir hikaye parçası sunar ve oyuncunun seçimleri (örneğin, "savaş" ya da "kaç") hikayenin sonunu değiştirir.

Yazılım Mühendisliği Açısından Değerlendirme:

Oyunumuz pygame ile geliştirilen bir 2D platform-macera oyunu olarak, yazılım mühendisliği prensiplerini uygulama ve ekip odaklı bir proje yürütme açısından değerli bir örnektir. Projenin teknik tasarımı, geliştirme süreci ve ekip yönetimi, yazılım mühendisliği disiplininin temel kavramlarını yansıtır. Maddeler halinde bu unsurlar ayrıntılı bir şekilde ele alınmıştır.

1. Teknik Tasarım ve Mimari Uygunluk

Modülerlik ve Kapsülleme:

Proje, **modüler** bir yaklaşımla tasarlanmıştır. Character, Platform, ve StoryScene gibi class'lar, her bir bileşenin işlevselliğini ayrı birimler halinde sınıflandırır. Örneğin, Character class'ı hareket, saldırı ve bakım mekaniklerini yönetirken, Platform class'ı ise çarpışma ve engel işlevselliğine odaklanır.

Sonuç olarak her modül tek bir sorumluluğa sahiptir, bu da kodun bakımını ve yeniden kullanılabilirliğini artırır.

Ölçeklenebilirlik (Scalability):

Tasarım, **ölçeklenebilir** bir yapı sunar. Bir karakterle başlasa da, Character class'ından miras alma (inheritance) yoluyla yeni karakter tipleri kolayca eklenebilir. Aynı şekilde, StoryScene modülü yeni hikaye dalları için genişletilebilir.

Yazılım mühendisliğinde, sistemlerin gelecekteki ihtiyaçlara uyum sağlaması kritik bir özelliktir. Proje bu yapı doğrultusunda geliştirilecektir.

Test Edilebilirlik (Testability):

Mekanikler (örneğin, move(), attack()) ayrı fonksiyonlar ve class'larla yazıldığı için **birim testi (unit testing)** uygulanabilir. Örneğin, Character class'ının zıplama mekanizmi bağımsız bir şekilde test edilebilir.

Test edilebilirlik, yazılım kalitesini doğrulama ve hata tespiti kolaylaştırma açısından temel bir mühendislik gereksinimidir; proje bu standardı karşılar.

2. Geliştirme Süreci ve Metodoloji

Iterative (Yinelemeli) Geliştirme:

Proje, 12 haftalık bir süreçte **iterative** bir yaklaşımla geliştirilecek.

Iterative süreç, geri bildirim döngülerini hızlandırır ve riskleri erken tespit ederek yazılımın evrimsel gelişimini destekler.

Proje geliştirilmesi küçük yinelemelerle (iterations) sağlanacak. İlk 1-2 hafta analiz ve temel tasarım, 3-10. haftalar uygulama, 11-12. haftalar ise test ve entegrasyon için ayrılmıştır. Projenin her safhasında geriye dönebilir ve proje yeniden düzenlenebilir.

Versiyon Kontrolü ve İş Birliği:

Git'in kullanımı, her ekip üyesinin haftalık commit'leri, değişikliklerin izlenmesini, çakışmaların çözülmesini ve kod geçmişinin korunmasını sağlar.

Versiyon kontrolü, yazılım mühendisliğinde ekip iş birliği ve kod bütünlüğü için vazgeçilmezdir; proje bu pratiği etkin bir şekilde uygular.

3.Yazılım Mühendisliği Prensiplerine Uygunluk

Kod Yeniden Kullanılabilirliği:

Örneğin, Character class'ındaki move() metodu tüm karakter tipleri tarafından paylaşılırken, attack() metodu her tip için özelleştirilebilirdir (polymorphism), farklı karakterler için değiştirilerek kullanılabilir. Bu, kod tekrarını azaltır.

Yeniden kullanılabilirlik, yazılım mühendisliğinde verimliliği artırır ve bakım maliyetlerini düşürür.

Hata Toleransı:

Proje hata toleransı gözetilerek yapılmıştır.

Enerji ve mutluluk gibi mekanikler, sınır kontrolleriyle (örneğin, max(0, energy)) desteklenir, böylece beklenmedik durumlarda oyun çökmez.

Hata toleransı, sistemin güvenilirliğini artırır ve kullanıcı deneyimini korur.

Performans Optimizasyonu:

Pygame'in 60 FPS döngüsü (clock.tick(60)), oyunun akıcı çalışmasını sağlar. Gereksiz hesaplamalar kullanılmaz (örneğin, her frame'de durum güncellemesi yerine zamanlayıcı kullanımı), böylece performans korunur.

Performans, yazılım mühendisliğinde kullanıcı memnuniyeti için kritik bir faktördür; bu proje temel optimizasyonları içerir.

4. Ekip Yönetimi ve Proje Uygulanabilirliği

Görev Bölüşümü ve Paralel Geliştirme:

Proje 5 kişilik ekip ile paralel geliştirme şeklinde ilerleyecektir: bir üye karakter mekaniklerini, bir diğeri platformları, bir başkası hikaye sistemini geliştirebilir.

Yazılım mühendisliğinde, kaynakların verimli kullanımı ve görevlerin dağıtımı, proje başarısını artırır.

Risk Yönetimi:

Iterative süreç, riskleri erken tespit eder (örneğin, zıplama mekaniği çalışmazsa ilk haftalarda fark edilir). Ayrıca, pygame gibi öğrenmesi kolay bir kütüphane seçimi, teknik riskleri azaltır.

Risk yönetimi, yazılım projelerinde belirsizlikleri minimize eder; proje bu açıdan dengeli bir yaklaşım sunar. Proje risk yönetimi ekseninde

5. Kullanılan Teknolojiler ve Uygunluk

Python: Projede kullanılacak programlama dilidir. OOP desteğiyle, projenin modüler ve okunabilir olmasını sağlar.

Pygame: 2D oyunlar için hafif ve esnek bir kütüphane; öğrenme eğrisi düşüktür ve 12 haftalık süreye uygundur.

JSON: Veri kalıcılığı için basit ve etkili; yazılım mühendisliğinde veri yönetimini kolaylaştırır.

Git: Ekip çalışmasını destekler, kod değişikliklerini takip eder.

Time: Zamanla değişen mekanikleri (örneğin enerji azalması, mutluluk hormonu artışı) yönetir.

Üst Mimari Tasarım

Proje, Model-View-Controller (MVC) tabanlı bir üst mimariyle tasarlanacaktır. Bu yaklaşım, iş mantığını, görselleştirmeyi ve kullanıcı etkileşimlerini ayırarak sistemin düzenli, esnek ve sürdürülebilir olmasını sağlar.

Bileşenler ve Roller:

Model (Oyun Mantığı):

Amaç: Oyunun temel iş mantığını yönetir.

Class'lar:

Character: Hareket, saldırı ve bakım mekaniklerini kapsar.

Platform: Engelleri, zıplama alanlarını, zemini ve çarpışmaları tanımlar.

StoryScene: Hikaye akışını ve seçimleri kontrol eder.

Uygunluk: Dış kütüphanelerden bağımsızdır; Python'un doğal dinamikleri ile yazılır, yeniden kullanılabilir ve test edilebilirdir.

View (Görselleştirme):

Amaç: Oyunun görsel çıktısını üretir.

Fonksiyonlar: draw_character(), render_screen() gibi Pygame metodlarını , sprite'ları ve arayüzü çizer.

Uygunluk: Görsellik, mantıktan ayrılmıştır; bu işlem, UI değişikliklerini kolaylaştırır.

Controller (Kontrol):

Amaç: Kullanıcı girdilerini alır ve mantığı günceller.

Yapı: Ana oyun döngüsü , klavye olaylarını ve zamanlayıcıları içerir.

Uygunluk: Kullanıcı etkileşimleri merkezi bir yerde toplanır, bu da kontrol akışını sade ve yönetilebilir kılar.

Ekip Yapısı ve Görev Dağılımı:

Eren Kuşdil (Karakter Mekanikleri): Character class'ı, hareket, saldırı, bakım mekanikleri ve karaktere eklenecek ekstra özellikler ile ilgilenir.

Berat Kayar (Platform Tasarımı): Platform class'ı, çarpışma algılama, seviye düzeni ve karaktere düşman botlar gibi özelliklerle ilgilenir.

Özer Koca (Hikaye Geliştiricisi): StoryScene class'ı, hikaye sahneleri, seçimler ve oyun akışı gibi özelliklerle ilgilenir.

Alperen Altunel (Arayüz ve Veri): Pygame ile sprite'lar, animasyonlar, ekran çizimi ve JSON veri yönetimi gibi özelliklerle ilgilenir.

Alaattin Uysal (Test ve Entegrasyon): Mekanik testi, oyun motoru, menü tasarımı, hata ayıklama, modül entegrasyonu ve performans kontrolü gibi özelliklerle ilgilenir. Codemaker ve Takım lideri.

KAYNAKÇA:

Game Development Stack Exchange Community. (t.y.). *Game Development Stack Exchange*. <https://gamedev.stackexchange.com/>

Pygame Community. (2023). *Pygame Documentation*. <https://www.pygame.org/docs/>

Real Python. (2021). *Pygame: A Primer*. <https://realpython.com/pygame-a-primer/>

Reddit Pygame Community. (t.y.). *r/pygame*. <https://reddit.com/r/pygame/>

Stack Overflow Community. (t.y.). *Stack Overflow*. <https://stackoverflow.com/>

Sweigart, A. (2012). *Making Games with Python & Pygame*. CreateSpace Independent Publishing Platform.

